

Degradation Contracts

Specifying and Enforcing How Software Behaves Under Pressure

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-004 v1.0 • series II

October 7, 2026 • cc by 4.0

Abstract — Every piece of software has two behaviors: the one it exhibits when resources are abundant, and the one it exhibits under pressure. The first is engineered — specified, tested, monitored, budgeted. The second is emergent: accidental, untested, invisible to buyers and often to authors. When memory runs low, bandwidth collapses, the battery fades, or the CPU throttles, modern applications degrade in ways nobody chose, nobody documented, and nobody can audit. We argue this asymmetry is a first-order engineering gap, not folklore to be tolerated. We propose **degradation contracts**: a declarative formalism in which an application specifies, per resource-pressure condition, an ordered ladder of fidelity tiers for each feature, a safe shedding order across features, hysteresis and recovery semantics, and floor invariants; and a runtime — a **pressure broker** — that measures pressure (in the spirit of Linux Pressure Stall Information), mediates directives to applications, and verifies compliance, producing a queryable degradation ledger. We define a formal model, state safety, liveness, and stability properties, sketch a reference architecture and DSL, design an evaluation, and confront the strongest objections: that applications will lie, that contracts across layers are unverifiable, that nobody will write manifests, and that enforcement overhead exceeds value. The gap is real and current: adjacent primitives (PSI, cgroups, Kubernetes eviction, Windows EcoQoS, Android ADPF, adaptive bitrate ladders, load shedding) each solve one slice, and nothing unifies them into a general discipline. The payoffs compound in the AI era, where quality is already a runtime variable — context truncation, model substitution, speculative accuracy — negotiated invisibly; contracts would make that negotiation inspectable, comparable, and fair.

p-rick working paper P-004 · series II (systems) · draft 1.0

Every piece of software has two behaviors: the one it exhibits when resources are abundant, and the one it exhibits under pressure. The first is engineered — specified, tested, monitored, budgeted. The second is emergent: accidental, untested, invisible to buyers and often to authors. When memory runs low, bandwidth collapses, the battery fades, or the CPU throttles, modern applications degrade in ways nobody chose, nobody documented, and nobody can audit. We argue this asymmetry is a first-order engineering gap, not folklore to be tolerated. We propose **degradation contracts**: a declarative formalism in which an application specifies, per resource-pressure condition, an ordered ladder of fidelity tiers for each feature, a safe shedding order across features, hysteresis and recovery semantics, and floor invariants; and a runtime — a **pressure broker** — that measures pressure (in the spirit of Linux Pressure Stall Information), mediates directives to applications, and verifies compliance, producing a queryable degradation ledger. We define a formal model, state safety, liveness, and stability properties, sketch a reference architecture and DSL, design an evaluation, and confront the strongest objections: that applications will lie, that contracts across layers are unverifiable, that nobody will write manifests, and that enforcement overhead exceeds value. The gap is real and current: adjacent primitives (PSI, cgroups, Kubernetes eviction, Windows EcoQoS, Android ADPF, adaptive bitrate ladders, load shedding) each solve one slice, and nothing unifies them into a general discipline. The payoffs compound in the AI era, where quality is already a runtime variable — context truncation, model substitution, speculative accuracy — negotiated invisibly; contracts would make that negotiation inspectable, comparable, and fair.

1 Introduction

Software behavior under abundance is the best-engineered artifact of our industry. Specifications enumerate features; tests freeze them; observability platforms watch them; SLOs budget their latency and error rate. Behavior under scarcity — memory pressure, thermal throttle, dying battery, collapsing bandwidth, CPU contention — has no comparable apparatus. It is decided, if at all, by scattered conditional statements, OOM killers, silent feature flakiness, and user confusion. The result is a universal, familiar failure pattern: the application that “just gets weird” when the machine is tired. Animations stutter, buttons deaden, saves lag, syncs corrupt, caches thrash — and no artifact anywhere says what should have happened instead.

This is not a small corner of behavior. Resource scarcity is the *common case* for most of the world’s devices: mid-range Android phones (the global median device) live under permanent memory and battery pressure; laptops throttle on lap use; every network is intermittently terrible. Industry practice treats abundance as the norm and scarcity as an incident. The truth is the reverse — pressure is a steady-state condition, and graceful behavior under it is a feature users can feel but cannot specify, buy, or audit.

Two industries have quietly proven that degradation can be a first-class engineering object. Video streaming made degradation *ladder-shaped and declarative*: an adaptive stream encodes a staircase of quality levels, and the player picks rungs by measured throughput — the single most successful degradation contract in production, though nobody calls it that. Web infrastructure made degradation *policy-shaped*: load shedding, admission control, and brownout-mode systems decide which requests to serve, drop, or simplify under overload. Mobile operating systems are inching the same direction: Windows EcoQoS lets processes request efficiency scheduling; Android’s ADPF lets games declare performance targets and receive hints; Linux exposes Pressure Stall Information so userspace can *see* pressure.

What does not exist — anywhere, in research or product — is the unification: a general language in which any application **declares how it degrades**, and a runtime that **measures pressure, dispatches degradation directives, enforces floors, and audits compliance**. We call the declaration a *degradation contract* and the runtime a *pressure broker*. This paper specifies both, argues the gap is load-bearing rather than incidental, and designs the system that would close it.

The thesis in one sentence: **degradation is the only user-visible behavior that no software system specifies, and it can be made contractual.**

2 The problem: degradation is emergent, accidental, and untested

Consider what actually happens when a modern laptop hits memory pressure. The kernel reclaims caches, then swaps, then invokes a killer (kernel OOM or userspace like oomd) that terminates some process by heuristic. Applications receive, at best, an `onTrimMemory` callback or a looming `SIGKILL`. None of them declared what they would sacrifice when memory grew scarce: which caches to drop first, which features to suspend, what fidelity floor keeps the app *usable*. Each developer improvised, usually by ignoring the condition entirely. The user experiences the composite of dozens of uncoordinated improvisations: a machine that becomes slow and strange together, with no way to ask “what was sacrificed, and by whom?”

The same story repeats per resource. Under thermal throttling, the OS slows the CPU and the application discovers it only through frame deadline misses — if it notices at all. Under battery saver, the OS imposes policy globally while applications make their own random trade-offs. Under bandwidth collapse, each app invents its own timeout ladder, retry policy, and “offline mode,” which is why the same train tunnel produces different behavior in every app simultaneously. Under CPU contention, schedulers fairly share cycles while *user value* — which feature of which app the human currently needs — is nowhere represented in the decision.

Four properties make this a *systems* problem rather than an accumulation of app bugs:

1. **Coordination.** Degradation decisions interact across processes. A browser shedding a tab and an OS killing a tab and a sync engine pausing uploads all fight for the same memory headroom. Uncoordinated, they overshoot (too much shed) or undershoot (too little), and they flap.
2. **Ordering.** Which feature dies first is a *product decision* with safety implications (the messaging app’s emergency SMS path must not be shed before the animation polish) — yet no artifact records the intended order, so it cannot be reviewed, tested, or honored.
3. **Recovery.** Systems that degrade must also *heal*, with hysteresis; oscillating between tiers (the “flap” pathology) can be worse than staying degraded. Recovery semantics are absent from every API that exists today.
4. **Accountability.** When a user asks “why is this slow?”, no system can answer with a degradation audit: what was shed, when, under what pressure, by whose decision. Observability covers latency and errors — the *symptoms* — never the *decisions*.

We name the accumulated, unrecorded liability of all this improvisation **degradation debt** — the analogue of technical debt for behavior under pressure. It is invisible because nothing measures it, and it grows silently as apps gain features (more things to degrade badly) and devices age (more pressure to degrade under).

3 The design space: fourteen adjacent systems, none at the center

The claim “nothing exists” must survive a landscape survey. We organize adjacent work by which slice of the problem it solves. (Citations in §12.)

****Per-resource pressure *measurement* (no semantics).** Linux Pressure Stall Information (PSI, kernel 4.20, 2018) reports per-resource stall time — the kernel finally made pressure *observable*. Facebook’s oomd consumes PSI to make userspace kill decisions. This is the sensing layer our proposal builds on; PSI has no notion of what applications *should* do, and oomd’s only directive is termination.

Infrastructure load shedding (server-side, request-granular). The “Tail at Scale” discipline (Dean & Barroso, 2013) and its descendants: Envoy’s overload manager, gRPC load-shedding policies, Netflix adaptive concurrency limits, admission control in databases (Oracle Resource Manager’s consumer groups come closest to contractual: it demotes sessions to lower resource share under contention — but only for CPU inside the database, with no application-visible semantics). These systems degrade *requests*, not *features*; the unit of shedding is an incoming call, chosen by the server. Nothing carries the notion of “the application’s own features degrade in a declared order.”

Brownout and control-theoretic degradation. The brownout line of research (Klein, Maggio, Árzen and colleagues, ~2014) reduced served content under overload using feedback control — a genuinely important precedent: degradation as a *controlled* variable. But brownout assumed a fixed two-tier request simplification inside a web server; no general model of per-feature ladders, ordering, or recovery, and no client/desktop story.

Streaming’s declarative ladders. MPEG-DASH and HLS encode explicit quality rungs; players traverse rungs on measured throughput with hysteresis. Per-title encoding (Netflix, 2015) tunes the ladder itself. This is the strongest existence proof of *declarative* degradation — and its limits: the ladder is one-dimensional (bitrate), single-resource (bandwidth), per-stream, and lives entirely in the media stack. Games independently invented the same idea as dynamic resolution scaling, again private to the render loop.

OS quality-of-service knobs (coarse, one-way). Windows EcoQoS (2022) marks processes “efficient”; Windows Power Throttling and macOS App Nap deprioritize; Android Doze and App Standby defer background work; Android’s ADPF lets games submit target frame durations and receive headroom hints — the closest any platform comes to *bidirectional* negotiation, but scoped to thermal/CPU headroom for games, with no feature semantics, no floors, no audit.

Adaptive middleware and imprecise computation (the research ancestors). The mid-2000s adaptive-QoS middleware line — Quality Objects (QuO, BBN/Washington University, Schmidt et al.) and its contemporaries — expressed QoS contracts between clients and objects in distributed settings; the imprecise-computation theory of the real-time community (J. Liu et al., 1994) split tasks into mandatory and optional parts with formal deadline semantics; anytime algorithms (Zilberstein, 1996) formalized monotone quality-versus-time trade-offs for AI planning. These are the true intellectual ancestors. They assumed distributed-object or real-time-scheduling settings, pre-cloud, pre-mobile, and none produced an OS-integrated, general-purpose discipline — they remain specialized theory.

Self-adaptive software and autonomic computing (the meta-framework). MAPE-K (Kephart & Chess, 2003) and the self-adaptive-software research program (de Lemos et al.) give the governing *loop* (measure–analyze–plan–execute) but deliberately leave the contract language, the enforcement point, and the cross-layer composition unspecified — the field’s own surveys list exactly this as the open problem.

Chaos engineering (tests failure, not degradation). Chaos engineering verifies that systems *fail* interestingly; it injects resource exhaustion and observes. It has no declarative target to verify *against* — it discovers your emergent degradation behavior and hopes it is acceptable.

The center of the design space — a general, declarative, per-feature, multi-resource degradation specification with a mediating runtime, floors, recovery, and audit — is empty. Each neighbor solved a slice,

validated the concept, and stopped at its domain boundary. The gap persists not because the idea is unknown but because no one owned the *unification*.

4 Why the gap persists

Four structural reasons, each with a counter-trend that is now dissolving it.

Ownership is split. Degradation decisions live half in the OS (which sees pressure) and half in applications (which own features). Neither can act alone: the OS cannot know that shedding the spell-checker before autosave is wrong; the app cannot know the system is under thermal pressure until too late. A shared contract *is* the missing coordination artifact. The trend dissolving this: OS vendors are already shipping pressure/negotiation APIs (PSI, EcoQoS, ADPF) — the substrate for a shared language now exists on every major platform.

Abundance bias in tooling. Performance engineering optimizes the p50/p99 of *throughput and latency* under load — abundance metrics. No dashboard displays “user-visible quality under pressure.” The trend: energy costs, mid-range global hardware, and battery-first design have moved pressure from incident to steady state; Apple’s Low Power Mode normalization proved users *notice and value* honest degradation.

Verification is genuinely hard. Testing degradation requires producing pressure states deterministically, which is why chaos tools exist; verifying *contract compliance* additionally needs a machine-checkable specification — which is precisely what does not exist, a chicken-and-egg the contract language breaks. The trend: runtime verification is mature (Leucker & Schallhart), and eBPF/ETW/EndpointSecurity make per-process resource behavior observable cheaply.

No economic surface. Users cannot ask “which app degrades best?” because no artifact distinguishes disciplined from undisciplined software; procurement cannot demand contracts because there is no format to demand. A standard manifest creates the market surface the same way nutrition labels did — and, closer to home, the way SLO/error-budget culture created the reliability market.

5 The degradation contract model

5.1 Resources, signals, and pressure states

Let the resource universe be finite: $\mathbf{R} = \{\text{memory, cpu, thermal, battery, bandwidth, disk, latency-to-critical-service}\}$. Each resource r has a *pressure signal* $\pi_r(t) \in [0,1]$, normalized so that 0 is abundance and 1 is exhaustion — PSI-style stall fractions for cpu/memory/io, thermal headroom fraction, battery drain-vs-charge margin, measured goodput against demand, and so on. Define a **pressure state** as the vector $\Pi(t) = (\pi_{\{r1\}}, \dots, \pi_{\{rk\}})$. To avoid pathological reactivity, signals are consumed as *tiered windows* (e.g., $\omega_r: [0,0.4) \rightarrow \text{L0 abundant}, [0.4,0.7) \rightarrow \text{L1 tight}, [0.7,0.9) \rightarrow \text{L2 scarce}, [0.9,1] \rightarrow \text{L3 critical}$), each with independent entry/exit hysteresis.

5.2 Features, tiers, and ladders

An application declares a set of features $\mathbf{F}$. Each feature $f \in \mathbf{F}$ carries a finite, totally ordered **fidelity ladder** $T_f = (f^0 > f^1 > \dots > f^{\{m_f\}})$, where f^0 is full fidelity and $f^{\{m_f\}}$ is *suspended* ($f^{\{m_f\}} = \perp$). Ladder rungs are the application’s semantic units — “animation at 60/30/no fps,” “sync full/incremental/deferred,” “voice hi-fi/telephone/off” — and only the application can define them; the runtime never invents semantics, only selects rungs.

5.3 The contract

A **degradation contract** is the tuple $C = (F, T, <, \rho, H, I, V)$:

- **F, T**: features and their ladders (above).
- **<**: the **shedding order**, a partial order over F encoding product safety: $f < g$ means *f must be degraded (or suspended) before g may be suspended*, i.e., g is protected relative to f . Acyclicity of $<$ is checkable at validation time; the transitive reduction is the reviewable artifact (“what dies first”).
- **ρ** : the **response map** $\rho: L(R) \rightarrow$ bounds, assigning to each pressure tier-vector an allowed fidelity bound per feature — formally a map from the pressure lattice to the product of ladders, specifying the *maximum degradation permitted* (the app may run better than the bound, never worse), plus *mandated* floors. We require ρ to be antitone in pressure: more pressure permits (weakly) lower fidelity.
- **H**: **hysteresis and recovery semantics**: per-tier entry/exit thresholds (enter L2 at 0.7, exit at 0.55), minimum dwell time per tier (anti-flap), and recovery delay τ_{rec} : after pressure clears, the app must return toward $\rho(\text{abundance})$ within τ_{rec} and at bounded ramp rate (avoiding quality whiplash).
- **I**: **invariants**, the formal properties the runtime checks: (i) *floor safety*: $\forall f \in \text{Floors: tier}(f) \geq \text{floor}(f, \Pi)$ — protected features never fall below their pressure-dependent floor; (ii) *liveness*: eventual recovery within τ_{rec} of sustained abundance; (iii) *stability*: at most k tier transitions per feature per sliding window (flap bound); (iv) *ordering compliance*: suspension of g only if every $f < g$ is already suspended.
- **V**: **violation semantics** — what the broker does when the app ignores directives: log-only (default), visual marking (UI badge “degraded contract: memory”), resource demotion (EcoQoS-style), or suspension. The default is transparency, not punishment: the point is an auditable record, and the market does the punishing.

5.4 Semantics and properties

The contract gives degradation a state-machine semantics: the app’s observable fidelity state $\sigma(t) \in \Pi_f$ evolves under broker directives; the broker computes directive targets from ρ , H and current Π . Three properties define correct behavior:

- **Safety**: $\sigma(t)$ never violates an active floor, and suspension ordering respects $<$. Violation is *detectable in bounded time* by the runtime monitor (each directive is acked or observable via resource telemetry).
- **Liveness**: if $\Pi(t) < \text{abundance-threshold}$ for τ_{rec} , then σ recovers to the abundance bound within $\tau_{\text{rec}} + \epsilon$.
- **Stability**: transition count per feature per window $\leq k$, enforced by H ’s dwell times.

These are deliberately modest — in the style of transactional memory’s forward-progress guarantees, not full verification. The aspiration is not to *prove* applications degrade correctly but to make the contract the *checkable interface* such that violations are recorded and blame is attributable.

5.5 Composition and cross-layer refinement

Contracts compose by **refinement**. A platform (OS shell, cloud runtime, browser) holds its own contract C_{plat} ; an app’s C_{app} must satisfy: floors of C_{plat} are respected, and C_{app} ’s lattices refine the platform’s per-signal tiers. This is where the model earns its generality: a cloud service’s broker can consume contracts from dozens of microservices and expose a single composed contract to its own load shedder (the “requests” the infra sheds become *feature tiers* of the composed service). A browser becomes a mini-broker for tabs; an OS the broker of last resort. The contract is the unit that travels across layers — the piece every prior system (server load shedding, DASH ladders, OS throttling) kept private.

6 Architecture: the pressure broker

A reference implementation on a modern OS:

1. **Pressure broker** (privileged daemon, or kernel service): consumes PSI/eBPF (Linux), ETW + Eco-QoS (Windows), EndpointSecurity/QoS classes (macOS), /proc + thermal + battery APIs; maintains tiered windows per resource with hysteresis; emits `PressureEvent(tier transitions)` globally.
2. **Contract registry**: validates manifests (`degradation.yaml`), checks $\leftarrow$ -acyclicity, lattice consistency, and floor coherence at install/update time; rejects or warns (validation is static and cheap).
3. **Per-process contract agent** (library, `libdgc`): receives directives `SetTier(f, k)` for each feature; the application implements tier functions; the agent acks with timing, exposing telemetry to the broker. Unmodified apps simply never register — no contract, no directives (they keep today’s behavior, but the broker still *records* their resource pressure exposure: degradation debt becomes measurable even for non-adopters).
4. **Enforcement hooks**: the broker maps pressure tiers to platform knobs — cgroup memory limits, Eco-QoS classes, ADPF hints — and maps app acks back; the *pairing* of directive and observable resource behavior is the compliance signal.
5. **Degradation ledger**: append-only local log of every pressure transition, directive, ack, and violation; exported as metrics (degradation-exposure seconds per resource; contract compliance rate; flap count) and queryable (“what was shed between 14:00 and 15:00 and why”) — the audit surface that does not exist today.
6. **Offline verification**: a contract model-checker for I-properties, runnable in CI: simulate tier traces, assert floors/ordering/liveness under adversarial pressure sequences — degradation chaos testing with a specification.

The DSL sketch (Appendix A) is deliberately boring: declarative YAML, one section per feature, tiers as names, ordering as a list. Boring is the point — contracts must be diffable in code review and greppable in audits.

7 What this buys, for whom

Users: an answer to “why is my machine weird right now” (ledger query); devices that degrade *honestly* — visible, ordered, recoverable; the ability to prefer disciplined software (the nutrition-label effect).

Developers: pressure as an API instead of folklore; tier functions are small, testable units; CI simulation of pressure traces replaces hoping; postmortems gain a degradation timeline.

Platform vendors: a differentiation axis that costs little (the substrate APIs already exist) and monetizes trust; reduced support load (“why is it slow” becomes answerable); a story for the AI-on-device era, where quality is negotiated per second (§8).

Procurement/enterprise: a vendor-requirement format (“all vendor apps must ship contracts with floors on sync/audit paths”), impossible to state today.

8 The AI era makes this more necessary, not less

AI systems are *natively* degradation-shaped. An assistant’s answer quality varies with context window truncation, model substitution (large $\rightarrow$ small $\rightarrow$ distilled), speculative decoding aggressiveness, retrieval depth, and quantization — all runtime quality knobs under memory/battery/latency pressure. Today these knobs are turned invisibly by whatever heuristic the vendor shipped; the user cannot see *which* quality they received, nor compare, nor hold anyone to a floor. Degradation contracts are precisely the interface that makes AI runtime quality **inspectable and contractual**: “under L2 memory pressure, this assistant answers with the 7B model and 4k context, *never below*, and the UI says so.” Agentic systems compound the need: agents that consume machine resources must negotiate with the pressure broker rather than heuristically grab; contracts are the negotiation protocol. Far from being obsoleted by AI, the formalism is the missing governance layer for it.

9 Prototype and evaluation design

A minimal credible prototype: the broker as a Linux userspace daemon (PSI + cgroups v2), libdgc with a C/Python binding, the YAML DSL, and the ledger as SQLite with a query CLI. Port three real applications: a video-streaming player (native ladders), a browser (tab eviction as tier moves), and a local LLM assistant (model/context tiers). Evaluate:

- **Compliance**: directive-ack latency; floor-violation rate ≈ 0 on ported apps; ledger completeness.
- **Stability**: transitions per feature-hour under synthetic pressure traces (compare flap vs. dwell-tuned H).
- **Utility under pressure**: task-completion proxies (e.g., words typed, commits, video minutes watched at $\geq$ declared tier) under memory/thermal squeeze, contract on vs. off — the headline number is *useful time under pressure*, not throughput.
- **Overhead**: broker CPU $\leq 1\%$ on mid-range hardware; manifest validation ≤ 50 ms.
- **Chaos validation**: reproduce 48 h of pressure traces from real telemetry; assert CI-simulated properties held in the wild.

Threats to validity: ported apps are enthusiast apps (selection bias); utility proxies are crude; single-platform prototype. The paper’s claim survives these: the contract format and broker semantics are the contribution; ports demonstrate feasibility.

10 Limitations and honest risks

Applications lie. An app can register a contract and ignore directives; floors are only as real as enforcement. Response: default violation semantics is *transparency* (ledger + UI marking), demotion for repeated violations; strongest enforcement reserved for platform-managed features (the OS can always evict memory, so floors implemented via platform knobs are physical). The honest concession: contracts are primarily *accountability* + *coordination*, secondarily coercion — the same bargain as every QoS standard ever fielded.

Cross-layer gaming. A composed platform contract is only as honest as its components. Response: ledgers compose; a platform exposing a composed contract must forward child telemetry or its own audit fails visibly. Concession: adversarial compositions need signed manifests — solvable, but real work.

Nobody writes manifests. The strongest objection. Response: three beachheads where writing is nearly free — media players (ladders already exist), games (dynamic-resolution logic already exists, ADPF already speaks performance), and local AI (model tiers already exist) — then let the ledger make non-adopters *visible* in their degradation debt. Chromium shipping a tab contract would move the industry more than any standard body. Concession: this is a beachhead theory of adoption, not a guarantee.

Semantic laundering. An app can define a ladder whose “lowest tier” is indistinguishable from full (declaring compliance trivially). Response: tier definitions are declarative and public; the market and reviewers can read them; platform certification can demand semantic tiers for protected classes (sync, autosave). Partial answer only — honesty about limits is part of the contract’s own design philosophy.

Energy paradox. The enforcement layer itself consumes energy. Response: PSI-style counters are already maintained by kernels; the broker is event-driven; measured target < 1% CPU. If that fails, the ledger degrades gracefully — itself a contract.

11 Roadmap

Phase 1 (this paper): model, DSL, broker design, three ports. Phase 2: cross-platform agents (Windows/macOS), composed contracts, browser tab-contract experiment. Phase 3: standardization path (an RFC-shaped spec; interest test with one OS vendor’s existing performance APIs), AI-quality contract extension. Open problems: formal verification of composed liveness; contract-aware scheduling (should the *scheduler itself* consider declared floors when placing work?); degradation-aware pricing (cloud instances that *bid* on quality floors); the game theory of shared-pressure bargaining between apps.

12 Conclusion

The software industry has spent fifty years specifying and verifying behavior under abundance while leaving behavior under pressure to folklore — an asymmetry that made sense when resources were growing exponentially and makes none now that the median device is permanently pressed and the newest workloads degrade by design. Every ingredient for closing the gap exists and is battle-tested in isolation: PSI measures pressure, cgroups and EcoQoS enforce shares, DASH proved declarative ladders, brownout proved control-theoretic shedding, imprecise computation and anytime algorithms supplied the formalism, chaos engineering supplied the test harness. What is missing is the unifying contract — the language that makes degradation reviewable, floors enforceable, recovery bounded, and debt auditable. Degradation contracts are that language. The first platform that ships a pressure broker with a real ledger will not merely make its apps behave better when the machine is tired; it will create the first honest market for behavior under scarcity, one nutrition label at a time.

Appendix A — DSL sketch

```

contract: com.example.editor/1
resources: [memory, cpu, battery]
features:
- id: animations
  ladder: [full, reduced, suspended]
- id: spellcheck
  ladder: [full, suspended]
- id: autosave
  ladder: [immediate, deferred-10s] # no suspended rung: never off
  shed-order: [animations, spellcheck] # autosave protected by absence
floors:
  autosave: deferred-10s # under any pressure <= L3
  spellcheck: L2: full # held until scarce
response:
  memory: L1: animations: reduced, L2: spellcheck: full, animations: suspended, L3:
  autosave: deferred-10s
  hysteresis: entry: L2: 0.70, exit: L2: 0.55, dwell: 30s, recover: 10s
  violations: log # log | mark | demote | suspend

```

Disclosure

p-rick is an independent research program. No vendor has commissioned or reviewed this work. The author has no financial interest in any system cited. Prior p-rick papers (P-001..P-003) concern personal-data infrastructure; this series-II paper shares no topic overlap — it concerns systems behavior under resource scarcity.

References

1. Barroso, L., & Dean, J. (2013). *The Tail at Scale*. Communications of the ACM 56(2).
2. Dean, J., & Barroso, L. *Hedged requests* — in [1], §“Taming tail latency”.
3. Weiner, J. (2018). *PSI — pressure stall information for CPU, memory, and IO*. Linux kernel documentation; LWN.net coverage, 2018.
4. Facebook Engineering (2018). *oomd: a more flexible userspace OOM killer* (and follow-up, 2019: *oomd 2.0*).
5. Klein, C., Maggio, M., Árzen, K.-E., et al. (2014). *Brownout: reducing the peak load with graceful performance degradation* (brownout line; see also Maggio et al., IEEE TSE follow-ups).
6. MPEG-DASH: ISO/IEC 23009-1; Stockhammer, T. (2011). *Dynamic adaptive streaming over HTTP*. ACM MMSys.
7. Netflix Technology Blog (2015). *Per-title encode optimization*.
8. Microsoft (2022). *EcoQoS — quality of service class for efficiency*. Windows developer blog; Microsoft (2017) *Power throttling*.
9. Android Developers (2022–). *Android Dynamic Performance Framework (ADPF) — performance hint API*.
10. Zinky, J., Bakken, D., & Schantz, R. (1997). *Architectural support for quality of service in CORBA* (Quality Objects / QuO line). OOPSLA.
11. Liu, J. W. S., Lin, K.-J., Shih, W.-K., et al. (1994). *Imprecise computations*. Proceedings of the IEEE 82(1).
12. Zilberstein, S. (1996). *Using anytime algorithms in intelligent systems*. AI Magazine 17(3).

13. Kephart, J., & Chess, D. (2003). *The vision of autonomic computing*. IEEE Computer 36(1).
14. de Lemos, R., et al. (2013). *Software engineering for self-adaptive systems: a second research roadmap*. Springer LNCS.
15. Leucker, M., & Schallhart, C. (2009). *A brief account of runtime verification*. Journal of Logic and Algebraic Programming 78(5).
16. Rosenthal, C., & Jones, N. (2017). *Chaos Engineering*. O'Reilly.
17. Meyer, B. (1988). *Object-Oriented Software Construction (design by contract)*. Prentice-Hall.
18. Huang, Y., Kintala, C., Kolettis, N., & Fulton, N. (1995). *Software rejuvenation: analysis, module and applications*. IEEE FTCS-25.
19. Kubernetes documentation (2024). *Pod QoS classes; PriorityClasses; resource eviction*.
20. Linux kernel (2024). *cgroup v2 — memory controller, memory.reclaim*.
21. Intel (2015). *Energy-efficient scheduling* / Android (2015). *JobScheduler, Doze, and App Standby*.
22. Kleppmann, M., et al. (2019). *Local-first software: you can own your data, in spite of the cloud*. Ink & Switch (offline degradation modes, cross-referenced in §3).